

Custom Annotations and AOP Binding in Spring — Quick Revision

Intro — What & How

A method can declare **what behaviour it needs** (via annotation), while the aspect defines **how that behaviour works**. Custom annotations cleanly connect business methods to cross-cutting concerns like logging, auditing, tracking time, rate limiting, retries, etc.

1. Separating "What" from "How"

Mixing execution-time-measuring code inside a business method gives it two responsibilities. Move that timing logic into an `@Around` aspect, and let the method just carry `@TrackExecutionTime` — this is **declarative programming**: method declares intent, doesn't implement it.

2. A Custom Annotation Does Nothing by Itself

An annotation is just **metadata** attached to code — nothing happens automatically when you apply it. Some other component (here, a Spring AOP aspect) must detect the annotation, read it, and actually perform the behaviour.

3. Creating a Custom Annotation

Declared with `@interface` (not `interface`). Needs **meta-annotations** to tell Java where it can be used and how long it lives — mainly `@Target` and `@Retention` (plus optional `@Documented`).

4. Understanding @Target

Defines **where** the annotation can legally be applied (method, class/type, field, parameter, constructor, or another annotation). For method-level Spring AOP,

`@Target(ElementType.METHOD)` is the standard choice. Can combine multiple targets using `{}`.

5. Understanding @Retention

Defines **how long** the annotation info is kept. `SOURCE` = discarded after compile. `CLASS` = kept in .class file but not visible at runtime (Java's default). `RUNTIME` = available via reflection while app runs — **required for Spring AOP** to detect it.

6. Understanding @Documented

Just tells Javadoc tools to include the annotation in generated docs. Has **no effect** on AOP matching or proxy creation — purely cosmetic/documentation.

7. Marker Annotations and Configured Annotations

A **marker annotation** has no properties — just flags "apply this behaviour." A **configured annotation** carries values (e.g. `@operation()`, `@warnAfterMillis()` with defaults) so it acts as both a marker and a mini config object that the aspect can read.

8. The `@annotation()` Pointcut Designator

Two forms: **Type-matching** — `@annotation(fully.qualified.Name)` just checks if the annotation is present. **Binding form** — `@annotation(variableName)` both matches AND passes the actual annotation instance into the advice method as a parameter (name must match the parameter name), letting you read its properties.

9. Complete Example: Measuring Execution Time

3-step recipe: (1) create annotation with `@Target(METHOD)` + `@Retention(RUNTIME)` + `@Documented`, (2) apply it to a service method with actual property values, (3) write an `@Around` aspect using `@annotation(trackExecutionTime)` binding form to read properties and measure/log execution time using `joinPoint.proceed()` inside try-finally.

10. Where Spring AOP Enters the Bean Lifecycle

Bean creation stages: read definition → instantiate → inject deps → Aware callbacks → BeanPostProcessor (before init) → init callbacks → BeanPostProcessor (after init) → final bean exposed. Spring AOP's auto-proxy creation is built on this **BeanPostProcessor** infrastructure — it checks if any advisor matches the bean and wraps it in a proxy if so.

11. BeanPostProcessor from First Principles

An extension point letting infra code inspect/modify beans during creation. Has `postProcessBeforeInitialization()` (runs before `@PostConstruct` etc.) and `postProcessAfterInitialization()` (runs after init callbacks — this is where Spring AOP normally wraps the bean into a proxy and returns that proxy as the final reference instead of the raw bean).

12. How Spring Decides Which Beans Need Proxies

A pointcut + advice combo = an **advisor**. As each bean is created, Spring checks if any advisor's pointcut matches that bean's class/methods. If yes → creates a proxy for it. If no → bean stays unproxied. Same logic applies for annotation-based advisors too.

13. The Container Stores the Final Reference

Once a proxy (P200) is created wrapping the target (T100), Spring stores/injects the **proxy**, not the raw target, into other beans. This is why DI and AOP work together — Spring controls object creation, injection, and what reference gets handed out.

14. The Method Interceptor Chain

When multiple aspects match one method, they form a **chain** (like nested calls): Caller → Proxy → interceptor1 → interceptor2 → interceptor3 → Target method, then unwinds back in reverse order. Each interceptor can run code before/after the chain, or on exception.

15. What proceed() Actually Means

`proceed()` doesn't always call the target method directly — it means "**continue to the next interceptor in the chain.**" Only the last `proceed()` in the chain actually reaches the target method. If an `@Around` advice never calls `proceed()`, the chain stops there and the target method never runs — making `@Around` the most powerful advice type.

16. Controlling Aspect Order with @Order

When multiple aspects match the same method, `@Order` decides the sequence. **Lower value = higher precedence** (runs first/outermost). Execution goes in ascending order-value sequence, then unwinds in reverse — the lowest-order aspect is the outermost layer.

17. Important Proxy-Based AOP Boundaries

Custom annotation AOP only works when the call passes through the **Spring proxy**: (1) manually-created objects (`new X()`) bypass Spring entirely — must be a Spring-managed bean; (2) **self-invocation** (a method calling another method in the same class via `this`) bypasses the proxy, so the advice won't trigger; (3) **private methods** can't be proxied — annotations should go on externally-invoked, non-private methods.

18. Complete Mental Model

Startup: read bean defs → create targets → inject deps → init callbacks → AOP checks advisors → create proxies for eligible beans → store/inject proxy references. **Runtime call:** caller → proxy → matching interceptors → chain begins → `@Around` calls `proceed()` → target executes → chain unwinds in reverse → result/exception returns to caller.

19. Key Takeaways

Annotation = metadata only, no behaviour by itself. `@Target` = where it can be used. `@Retention(RUNTIME)` = required for Spring to see it at runtime. `@Documented` = docs only, no AOP effect. `@annotation(Type)` = match by type; `@annotation(varName)` = bind instance to advice param. Spring AOP proxies are created via `BeanPostProcessor` and injected as the final reference. A call must pass through the proxy for advice to fire. `proceed()` continues the chain (not always target directly). Lower `@Order` = higher precedence. Self-invocation & private methods break proxy-based AOP.